



Module 4 — Interaction avec les éléments (Cypress)



Objectifs du module

À la fin de ce module, l'étudiant saura :

- interagir avec une page web (cliquer, écrire, sélectionner)
 - vérifier l'état réel des éléments
 - comprendre **ce qu'il faut tester après une action**
 - écrire des tests utiles (pas des tests "vides")
 - respecter l'architecture du projet
-



Où créer les fichiers (rappel important)

Tous les fichiers de ce module doivent être créés ici :

```
cypress/  
└─ e2e/  
    └─ module4-interactions/  
        ├── exemples/  
        └── exercices/
```



Un fichier = une notion



Tous les fichiers finissent par `.cy.js`



Site de test utilisé (réel et fonctionnel)

Nous utiliserons le **site officiel de démonstration Cypress** :

`https://example.cypress.io`

Ce site est :

- stable
 - gratuit
 - prévu pour l'apprentissage
 - utilisé par Cypress lui-même
-

1 Cliquer sur un élément — `cy.click()`

◆ Principe

`cy.click()` permet de simuler un clic utilisateur.

Mais ⚠ cliquer seul ne suffit pas.

👉 On doit toujours vérifier un effet visible du clic.

❌ Mauvais exemple (à éviter)

```
cy.get(".action-btn")
  .click()
  .should("exist");
```

❌ Pourquoi c'est mauvais ?

- l'élément existait déjà
 - le clic n'a aucun effet vérifié
 - le test peut passer même si le clic ne sert à rien
-

✅ Bon exemple (utile)

📄 Fichier

```
cypress/e2e/module4-interactions/exemples/click.cy.js
describe("Interaction - clic", () => {
  it("Clique sur un bouton et vérifie l'effet", () => {
    cy.visit("https://example.cypress.io/commands/actions");

    cy.get(".action-btn").click();

    // Vérification de l'effet du clic
    cy.contains("Button clicked").should("be.visible");
  });
});
```

🧠 Ce qu'on vérifie

- le clic a provoqué un changement
- l'utilisateur voit quelque chose de nouveau

2 Écrire dans un champ — `cy.type()`

◆ Principe

`cy.type()` simule la saisie clavier de l'utilisateur.

Exemple

```
cyypress/e2e/module4-interactions/exemples/type.cy.js
describe("Interaction - saisie", () => {
  it("Tape un email dans un champ", () => {
    cy.visit("https://example.cypress.io/commands/actions");

    cy.get(".action-email")
      .type("test@email.com")
      .should("have.value", "test@email.com");
  });
});
```

Pourquoi c'est bien

- on agit
 - on vérifie que la valeur a bien changé
-

3 Sélectionner une option — `cy.select()`

◆ Principe

`cy.select()` permet de choisir une option dans un `<select>`.

Exemple

```
cyypress/e2e/module4-interactions/exemples/select.cy.js
describe("Interaction - select", () => {
  it("Sélectionne une option", () => {
```

```
cy.visit("https://example.cypress.io/commands/actions");

cy.get(".action-select")
  .select("apples")
  .should("have.value", "apples");
});
});
```

4 Vérifier l'état des éléments

◆ États courants

Assertion	Signification
be.visible	visible à l'écran
be.checked	coché
be.disabled	désactivé
exist	présent dans le DOM

Exemple checkbox

```
cypress/e2e/module4-interactions/exemples/checkbox.cy.js
describe("État des éléments", () => {
  it("Coche une checkbox", () => {
    cy.visit("https://example.cypress.io/commands/actions");

    cy.get(".action-checkboxes [type='checkbox']")
      .first()
      .check()
      .should("be.checked");
  });
});
```

5 Règle d'or à retenir (très importante)

 Après une action, on vérifie toujours un effet.

 Mauvais :

```
cy.click().should("exist");
```

 Bon :

```
cy.click();
```

```
cy.contains("Succès").should("be.visible");
```

Exercices pratiques



Exercice 1 — Cliquer et vérifier



Fichier

```
cypress/e2e/module4-interactions/exercices/exo_click.cy.js
```



Consigne

1. Ouvrir la page Actions
 2. Cliquer sur le bouton
 3. Vérifier qu'un message apparaît
-



Solution

```
describe("Exercice clic", () => {  
  it("Clique sur le bouton et vérifie le message", () => {  
    cy.visit("https://example.cypress.io/commands/actions");  
  
    cy.get(".action-btn").click();  
  
    cy.contains("This popover shows up on click").should("be.visible");  
  });  
});
```



Exercice 2 — Saisie utilisateur



Fichier

```
cypress/e2e/module4-interactions/exercices/exo_type.cy.js
```



Consigne

1. Ouvrir la page Actions
 2. Taper un email
 3. Vérifier que le champ contient la valeur
-

Solution

```
describe("Exercice saisie", () => {
  it("Tape un email et vérifie la valeur", () => {
    cy.visit("https://example.cypress.io/commands/actions");

    cy.get(".action-email")
      .type("student@test.com")
      .should("have.value", "student@test.com");
  });
});
```



Exercice 3 — Checkbox



Fichier

cypress/e2e/module4-interactions/exercices/exo_checkbox.cy.js

Consigne

1. Ouvrir la page Actions
 2. Cocher une checkbox
 3. Vérifier qu'elle est cochée
-

Solution

```
describe("Exercice checkbox", () => {
  it("Coche une checkbox", () => {
    cy.visit("https://example.cypress.io/commands/actions");

    cy.get(".action-checkboxes [type='checkbox']")
      .first()
      .check()
      .should("be.checked");
  });
});
```



FICHE MÉMO — Module 4 (À DONNER À L'ÉTUDIANT)

Commandes essentielles

Action utilisateur Commande Cypress

Cliquer `cy.click()`

Action utilisateur Commande Cypress

Taper du texte	<code>cy.type()</code>
Sélectionner	<code>cy.select()</code>
Cocher	<code>cy.check()</code>
Décocher	<code>cy.uncheck()</code>

📌 Assertions les plus utilisées

Assertion	Signification
<code>be.visible</code>	visible à l'écran
<code>have.value</code>	valeur exacte
<code>be.checked</code>	coché
<code>be.disabled</code>	désactivé
<code>exist</code>	existe dans la page

📌 Structure recommandée

```
module4-interactions/  
├── exemples/  
│   ├── click.cy.js  
│   ├── type.cy.js  
│   └── select.cy.js  
└── exercices/  
    ├── exercice_formulaire.cy.js  
    └── exercice_click.cy.js
```

👉 Un fichier = une idée

⚠️ PIÈGES CLASSIQUES DES DÉBUTANTS (TRÈS IMPORTANT)

❌ Piège 1 — Oublier de vérifier après une action

❌ Mauvais exemple

```
cy.get(".action-btn").click();
```

👉 On clique... mais on ne vérifie rien

✅ Bon réflexe

```
cy.get(".action-btn")
  .click()
  .should("exist");
```

🧠 Règle d'or

Toute action doit être suivie d'une vérification

❌ Piège 2 — Tester ce que l'utilisateur ne voit pas

```
cy.get("input").invoke("val");
```

- ❌ Mauvais pour un débutant
 - ❌ Pas lisible
 - ❌ Pas orienté utilisateur
-

✅ Préférer

```
cy.get("input").should("have.value", "test");
```

❌ Piège 3 — Tests trop longs

```
it("fait plein de choses", () => {
  // 50 lignes
});
```

- ❌ Difficile à lire
 - ❌ Difficile à corriger
-

✅ Bonne pratique

Un `it` = une intention claire

❌ Piège 4 — Sélecteurs trop compliqués


```
cy.get("div > ul > li:nth-child(3) > button")
```

- ✗ Fragile
- ✗ Illisible

✓ Préférer

```
cy.get(".action-btn")
```

🚧 Pourquoi on n'utilise PAS encore le POM ?

👉 C'est volontaire

À ce stade :

- on apprend comment Cypress interagit avec le navigateur
- on voit les sélecteurs
- on comprend les assertions

📌 Le POM arrivera plus tard pour :

- éviter les duplications
- améliorer la lisibilité
- structurer un projet réel

💬 Phrase de transition à dire à l'étudiant :

“On écrit volontairement les tests directement pour comprendre ce qu'on fait. Ensuite, on apprendra à organiser tout ça proprement.”

✓ Conclusion du Module 4

À ce stade, l'étudiant sait :

- cliquer
- taper
- sélectionner
- vérifier des états
- écrire des tests utiles

- éviter les assertions inutiles

👉 **Il est maintenant prêt pour écrire de vrais scénarios utilisateur.**